arXiv:2509.23651v3 [cs.RO] 25 May 2026

HeLoM: Hierarchical Learning for Whole-Body Loco-Manipulation by a Hexapod Robot

Xinrong Yang[1*], Peizhuo Li[1*], Hongyi Li[1,2], Yifeng Peng[1], Arhaan Jain[1], Junkai Lu[1], Linnan Chang[1], Yuhong Cao[1], Yifeng Zhang[1†], Ge Sun[1], Guillaume Sartoretti[1]

***Abstract*— In nature, animals often need to move/manipulate objects comparable in weight/size to their own bodies. Compared to grasping and carrying, pushing provides a more straightforward and efficient non-prehensile manipulation strategy, avoiding complex grasp design while leveraging direct contact to regulate an object's pose during interaction. Achieving effective pushing, however, requires both sufficient manipulation capability and stable whole-body coordination, which is particularly challenging when dealing with heavy or irregular objects. To address these challenges, we propose HeLoM, a learning-based hierarchical whole-body manipulation framework for hexapod robots that exploits coordinated multi-limb control and is applicable to multi-legged robotic systems. Inspired by the cooperative strategies of multi-legged insects, our framework leverages multiple contact points and high degrees of freedom to enable efficient and dynamic whole-body coordination during object interaction. HeLoM's high-level planner plans pushing behaviors, while its low-level controller maintains locomotion stability and generates dynamically consistent joint actions. This design enables the robot to maintain balance while executing continuous and controllable pushing behaviors through coordinated foreleg interaction and supportive hind-leg propulsion. We validate the effectiveness of HeLoM through both simulation and real-world experiments. Results show that our framework can stably push objects of varying sizes and unknown physical properties to designated goal poses in the real world.**

I. INTRODUCTION

Mobile manipulation addresses the integration of a dedicated mobile base with manipulation capabilities, allowing robots to perform tasks that demand coordinated movement as well as dexterous interaction [1]. Transporting heavy objects, such as delivery packages or toolboxes comparable in weight to the robot demands flexible and well-coordinated mobility and manipulation capabilities. In such scenarios, non-prehensile manipulation (e.g., pushing or sliding) often provides a more straightforward and capable solution than prehensile manipulation (e.g., grasping or carrying) [2], for many tasks that allow it. This approach eliminates the need for complex gripper design to instead rely on direct contact between the robot's body or limbs and the object to regulate its pose during movement. Legged robots are particularly well-suited for this form of manipulation. With their high degrees of freedom, they can coordinate limb motions through whole-body control to achieve efficient and continuous pushing behaviors throughout the task [3].

* These authors contributed equally to this work.
† Corresponding author: Yifeng Zhang (yifengz@nus.edu.sg)
[1] MARMoT Lab, Department of Mechanical Engineering, National University of Singapore, Singapore
[2] Center for X-mechanics, Zhejiang University, Hangzhou, China

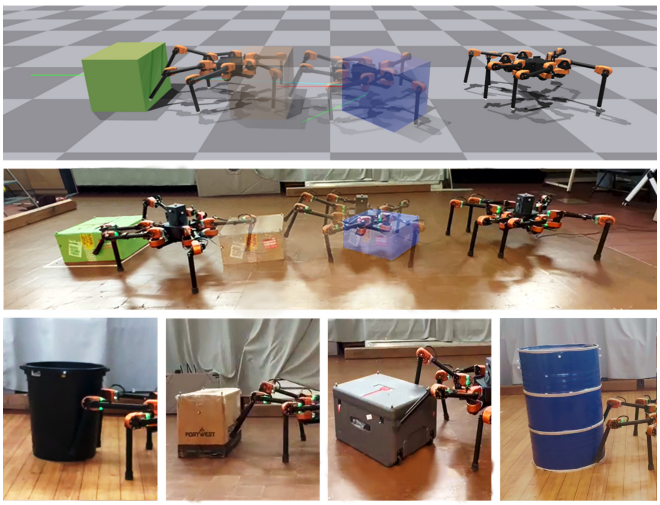


Fig. 1. Examples of our series-elastic hexapod robot pushing different objects to desired poses using its forelegs with our proposed **HeLoM** framework, both in simulation (top) and real-world experiments (bottom).

With the rapid development of learning-based locomotion and posture control in legged robots [4], researchers have begun extending these methods to manipulation tasks in order to achieve whole-body manipulation [5]. Early approaches relied on using the robot's body to directly contact and apply force to an object [3]. While this strategy is simple to implement, it often requires frequent adjustments of the robot's body position, resulting in inefficient transport and prolonged task duration. Moreover, impact forces are directly transmitted to the robot's actuators and sensors, posing risks of damage and reducing long-term reliability. To mitigate these issues, some studies introduced dedicated robotic arms, which enable more precise interactions and flexible force application through end effectors [6]. While this approach improves manipulation dexterity, contact is generally restricted to a single point, limiting the ability to deliver continuous and stable pushing forces to large objects. In addition, the inclusion of an arm inevitably increases system complexity, design cost, and control difficulty. Researchers have also explored pushing through direct contact with the feet (end effectors) of quadruped robots [7]. This method requires no additional hardware and naturally integrates locomotion with manipulation. However, with fewer feet grounded during pushing, the robot's overall stability is reduced, and the manipulation capability of individual legs becomes limited, which constrains the effectiveness of this approach.

Biological inspiration suggests that robots with more than four legs may offer clear advantages for manipulation. In-

sects such as hexapods and octopods exhibit remarkable coordination strategies when interacting with objects [8]. They can use a subset of their legs to maintain simultaneous contact, continuously adjust object orientation, and move at the same time, while the remaining legs provide balance and propulsion. In tasks such as dung-ball rolling, beetles further coordinate their whole body to maximize force application. Such multi-limb cooperation leverages redundant degrees of freedom and contact points to improve stability, precision, and manipulation performance.

Inspired by the manipulation strategies observed in multi-legged insects, we propose **HeLoM**, a reinforcement learning (RL)-based control framework for hexapod robots that can be extended to other multi-legged robotic systems, which exploits whole-body coordination to accomplish large object pushing tasks. Our framework adopts a hierarchical structure consisting of a *Planner* and a *Controller*. Our *Planner* directs commands to the *Controller* to drive coordinated dual-foreleg pushing behaviors, with the goal of moving the object progressively closer to its target position and orientation. Meanwhile, our *Controller* focuses on accurately executing these commands by coordinating all joints to maintain balance, handle external disturbances, and optimize torque distribution, as illustrated in Fig 1. Through this process, the robot achieves stable whole-body movements and effectively executes pushing behaviors, ensuring that the *Planner*'s intended manipulation strategy is faithfully realized. In doing so, our work demonstrates the potential of hexapod robots in challenging manipulation tasks and offers a promising pathway toward whole-body manipulation with multi-legged platforms. Thus, our main contributions are:

- We propose **HeLoM**, a bio-inspired hierarchical RL framework for whole-body loco-manipulation in multi-legged systems. Our approach reformulates loco-manipulation as learning whole-body interaction behaviors, instead of decoupling locomotion and manipulation. To the best of our knowledge, it is the first learning-based realization of whole-body loco-manipulation on a large hexapod robot, supporting more complex manipulation tasks in multi-legged robots.
- We train a high-level *Planner* and a low-level *Controller*, where the *Planner* outputs commands to the *Controller*, which unifies locomotion and manipulation to realize coordinated whole-body behaviors. By leveraging the high degrees of freedom and multiple contacts of multi-legged systems, our approach enables efficient and continuous pushing without requiring prior knowledge of object properties.
- We validate the effectiveness and robustness of our approach through extensive experiments on a real hexapod robot, addressing the sim-to-real gap and demonstrating policy generalization to objects of varying sizes, shapes,and out-of-distribution (OOD) scenarios.

II. Related Works

A. Legged Manipulation Controller

Recent interest in legged robots has expanded from locomotion to manipulation, leading to the exploration of various control approaches. Optimal control has been applied to legged robot pushing, including pushing with the body [9], pushing with a single dedicated arm [10], [11], and pushing with two arms of a centaur-like robot [12]. However, these optimal control-based approaches rely on precise object models, which must be assumed or estimated through costly and often inaccurate identification. In addition, convexifying complex loco-manipulation problems typically requires simplifying robot dynamics [13], introducing control errors and limiting versatility in contact-rich tasks. To address these limitations, recent work has turned to fully RL-based controllers for legged manipulation. Ji et al. trained a dynamic loco-manipulation policy that enables a quadruped to dribble a ball while running over rough terrain [14]. Other studies focus on legged robots with dedicated arms: Fu et al. and Portela et al. developed whole-body controllers on arm-mounted quadruped platforms, achieving manipulation in the position and force domains, respectively [15], [16]. Pushing the boundaries of limb utilization, Zhu et al. introduced the ReLIC framework to achieve flexible interlimb coordination, enabling the quadruped to dynamically assign its legs for either locomotion or assisting the arm in manipulation [17]. Building upon such complex loco-manipulation systems, Ha et al. further enhanced their autonomy by introducing a task-centric high-level policy [18].

Nevertheless, most prior work focuses on quadrupeds, while multi-legged robots remain underexplored. With more legs, hexapod robots provide enhanced stability under external reaction forces in contact-rich tasks [19], even when two legs are engaged in manipulation. Existing studies, such as using Bézier trajectories for obstacle manipulation [20] and hexapods grasping objects [21], generally treat the body as a fixed base and overlook body–leg coordination, leaving much of their potential for agile loco-manipulation untapped. While a recent multitask framework achieves simultaneous loco-manipulation [22], it still lacks autonomous planning capabilities.

B. Legged Robot Pushing

Research on legged robot pushing has investigated a variety of approaches. Rigo et al. employed hierarchical MPC to optimize contact forces and locomotion for planar box pushing with the robot body, but their method relies on accurate object models and flat terrain [23]. Improving on this, Sombolestan and Nguyen incorporated online estimation of unknown mass and friction, enabling quadrupeds to push objects on slopes; however, their framework only models the robot's dynamics and does not explicitly regulate the object's pose [9]. Jeon et al. advanced body-based pushing by training whole-body repositioning policies through hierarchical control and deep latent embeddings, achieving high success rates even with heavy loads. Yet, their controller's effectiveness

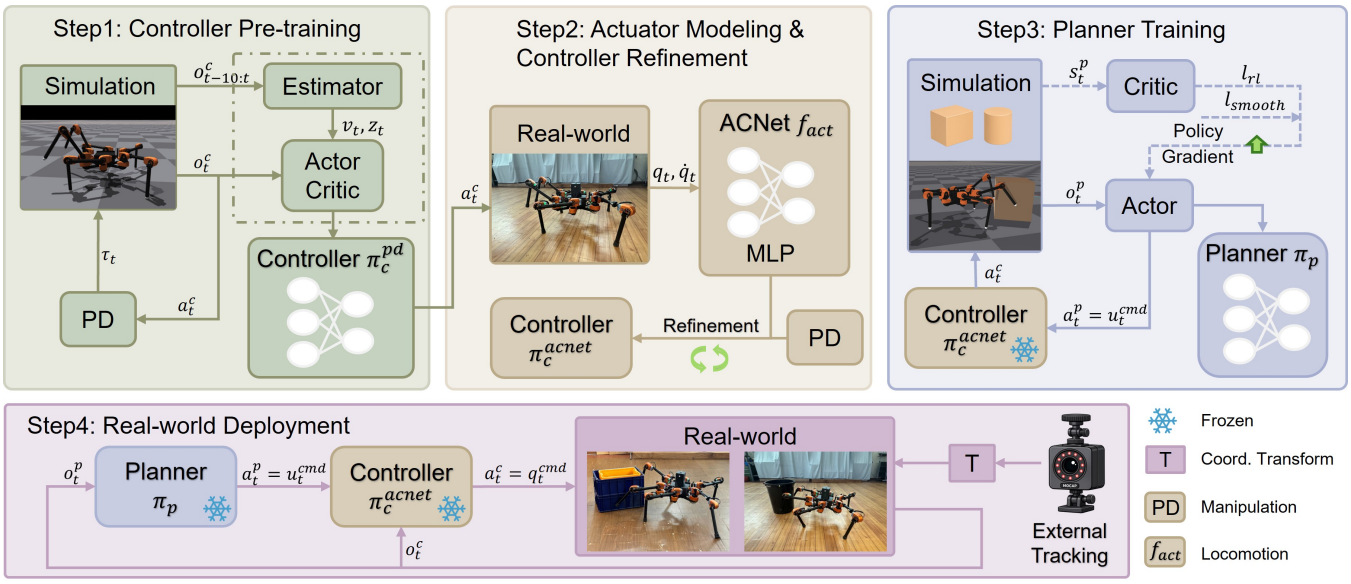


Fig. 2. **Overview of the proposed HeLoM framework**. During training, (1) we first pre-train the *Controller* with randomly sampled commands and a PD controller to establish basic loco-manipulation capabilities. (2) We then collect real-world data using the pre-trained controller, train an actuator network, and replace the PD controller to further refine the *Controller* with learned actuator dynamics. (3) Next, the *Controller* is frozen, and the *Planner* is trained on top of it to generate effective pushing commands. To improve smoothness and coordination, our *Planner* and *Controller* are executed synchronously at 50 Hz. (4) During deployment, a motion-capture system provides the robot and object states in the world frame, followed by coordinate transformation.

remains constrained by the body-pushing paradigm, which limits the robot's workspace and the directions in which forces can be applied [3]. Recognizing the drawbacks of body pushing, Arm et al. shifted the focus to leg-based pushing, where a robust RL controller enables individual legs to perform diverse non-prehensile tasks. This approach expands the contact point's workspace, nevertheless, high-level trajectory planning still depends on human teleoperation [24]. Dadiotis et al. proposed a constrained RL policy as a high-level controller for quadrupeds equipped with arms, allowing adaptive selection of contact points to prevent toppling [6]. However, their method requires multiple pushes to achieve object repositioning and reorientation. To overcome the reliance on repeated contacts, Hong et al. introduced a multi-agent system in which two quadrupeds collaborate to push large objects [25]. Although effective, the cost of multiple robots and the reliability of robot communication remain a concern. In a different direction, Leung et al. drew inspiration from dung beetles, adapting hexapod leg movements to roll balls across varied terrains. Nonetheless, their study focuses on ball rolling and does not generalize to broader planar pushing tasks [8].

III. Hierarchical Loco-Manipulation Learning

Our framework addresses the pushing task for hexapod robots by decomposing the problem into interaction planning and control execution. Both layers are formulated as RL problems: The *Planner* learns to produce pushing commands, while the *Controller* learns to translate them into feasible whole-body joint actions under contact constraints. An overview of the **HeLoM** architecture is provided in Fig. 2.

TABLE I

Planner Observation Representation and Notations.

Notations		Components	Dim	Components	Dim	Total
s_t^p	o_t^p	p_{object}	3	p_{object}^{cmd}	3	81
		R_{object}	3	R_{object}^{cmd}	3	
		o_t^c	69			
	x_t^p	$dimensions$	3	e_{object}	1	20
		m_{object}	1	$contact$	2	
		com	3	μ_{object}	1	
		I_{object}	9			

TABLE II

Planner reward terms and weights ($dt = 0.02$).

Term	Expression	Weight
$r_{\text{foot,obj}}$	$\frac{1}{2}\sum_{i\in\{l,r\}} \exp\left(-d_i/\sigma_{fo}\right)$	$1dt$
$r_{\text{foot,contact}}$	$\frac{1}{2}\sum_{i\in\{l,r\}} \exp\left(-[h_i - h^*]_+/\sigma_{fh}\right)$	$0.8dt$
$r_{\text{obj,tar}}$	$\alpha \exp\left(-\|p_{obj} - p_{tar}\|/\sigma_{tn}\right) + (1-\alpha)\left(1 + \|p_{obj} - p_{tar}\|/(d_{task}\sigma_{tf})\right)^{-1}$	$3dt$
$r_{\text{obj,dir}}$	$(v_{obj}^{xy} \cdot \hat{d}_{goal})\,\lambda_{\text{push}}$	$1.5dt$
$r_{\text{obj,yaw}}$	$\exp\left(-\theta_z^2/\sigma_\theta^2\right)\,\lambda_{\text{turn}}$	$1dt$
$r_{\text{obj,turn}}$	$\text{ReLU}(\omega_z \tanh(e_\theta^s/\sigma_e))\,\lambda_{\text{turn}}$	$0.4dt$
$r_{\text{obj,tilt}}$	$\|g_{obj}^{xy}\|^2$	$-10dt$
$r_{\text{termination}}$	$\mathbb{I}_{\text{finish}}$	$10dt$
$r_{\text{action rate}}$	$\|a_t^p - a_{t-1}^p\|^2$	$-0.5dt$
$r_{\text{action limits}}$	$\sum_j \left([a_j^{\min} - a_{t,j}]_+ + [a_{t,j} - a_j^{\max}]_+\right)$	$-1dt$

A. Pushing Planner

Our *Planner* determines the desired trajectories of the two front feet (end-effectors) along with body motions to generate effective interaction behaviors, guiding the object to its target position and orientation (yaw). Its RL formulation is introduced as follows:

1) Observations & Action Space: To endow our *Planner* policy with planning capability that accounts for the robot's

posture, we design the observation vector o_t^p. As summarized in Table I, this input not only includes the object pose in the base frame $p_{object} \in \mathbb{R}^3$, $R_{object} \in \mathbb{R}^3$, and the target pose $p_{object}^{cmd} \in \mathbb{R}^3$, $R_{object}^{cmd} \in \mathbb{R}^3$, but also incorporates the full observation vector o_t^c used by the low-level *Controller*, where o_t^c is defined as:

$$o_t^c = [\omega_t,\, g_t,\, q_t,\, \dot{q}_t,\, a_{t-1}^c,\, u_t^{cmd}]^T. \tag{1}$$

Here, $\omega_t \in \mathbb{R}^3$ denotes the base angular velocity, and $g_t \in \mathbb{R}^3$ is the gravity vector. The vectors $q_t \in \mathbb{R}^{18}$ and $\dot{q}t \in \mathbb{R}^{18}$ represent the joint positions and velocities measured by the encoders, while $a_{t-1}^c \in \mathbb{R}^{18}$ corresponds to the previous action of the *Controller*. The command vector $u_t^{cmd} \in \mathbb{R}^9$, which also serves as the previous output (i.e., action) a_{t-1}^p of the *Planner* actor network, includes both the desired base velocity $v_t^{\text{cmd}} = [v^{cmd}x, t, v^{cmd}y, t, \omega_{yaw,t}^{cmd}]$ and the desired positions of the two front feet in the base frame $p_t^{\text{cmd}} \in \mathbb{R}^6$.

In real-world settings, the physical properties of the manipulated object, such as its mass m_{object}, $dimensions$, center of mass com, inertia matrix I_{objec}, coefficient of restitution e_{objec}, and surface friction coefficient μ_{objec}, are typically unknown. Therefore, we treat these properties as privileged observations and provide them only to the critic network during training. Furthermore, we find that incorporating a binary indicator $contact$ into the critic network, which represents whether the two front feet are in contact with the object, improves the accuracy of value estimation. We empirically observed that this improvement provides more reliable feedback to the actor and facilitates more effective exploration during the early stages of training.

2) Reward Functions: To enable the robot to push various objects to target poses using its feet, we design a reward function composed of four groups of terms, which we describe below and in Table II:

Foot-object interaction: $r_{\text{foot,obj}}$ drives the feet toward the nearest surface of the object based on a distance metric. Compared to approaches that target the object center, this design allows the feet to adaptively adjust contact locations during interaction, enabling more precise and stable placement. $r_{\text{foot,contact}}$ encourages contacts at lower regions of the object, which improves pose stability and reduces the risk of tipping, especially under high-friction or low center-of-mass conditions. Furthermore, to enhance training stability, the environment is reset when the object's tilt angle around the x and y axes exceeds $40°$, thereby filtering out unstable states and accelerating convergence.

Goal-directed object motion: $r_{\text{obj,tar}}$ guides the object toward the target position. Considering that the target distance may vary over a wide range, we design two complementary distance-based terms to provide effective gradients at different stages. Specifically, $\exp(-\|p_{obj} - p_{tar}\|/\sigma_{tn})$ provides strong gradients when the object is close to the target, facilitating precise convergence, while $(1 + \|p_{obj} - p_{tar}\|/(d_{task}\sigma_{tf}))^{-1}$ provides a smoother and persistent signal at larger distances, reflecting the task progress and reducing inefficient exploration in early stages. $r_{\text{obj,dir}}$ encourages the object to move along the direction toward the target, guiding the robot to approach the goal along near-optimal paths and improving overall motion efficiency. In addition, $r_{\text{obj,yaw}}$ and $r_{\text{obj,turn}}$ jointly regulate the object orientation, ensuring that the desired pose is achieved upon reaching the target. In particular, $r_{\text{obj,turn}}$ provides stronger rotational guidance under large angular deviations, compensating for the diminished gradients of $r_{\text{obj,yaw}}$ in such cases, thereby mitigating unstable exploration when the orientation error is large.

Furthermore, we introduce weighting coefficients λ_{turn} and λ_{push}, defined as:

$$\lambda_{\text{turn}} = \text{clip}(\theta_z/\sigma_1,\, 0,\, 1), \quad \lambda_{\text{push}} = \exp(-(\theta_z/\sigma_2)^2) \tag{2}$$

where $\sigma_1 = 0.5$ and $\sigma_2 = 0.8$. These coefficients are designed as smoothly varying functions to enable a gradual transition from rotation-dominant to translation-dominant behaviors during training. If rotation and translation are optimized simultaneously with equal emphasis, the rotation phase may temporarily increase the distance to the target, thereby weakening distance-based rewards and leading to suboptimal local solutions.

Regularizing manipulation behavior: $r_{\text{orientation}}$ penalizes undesired object rotations around the x and y axes, which are often caused by unstable contacts or collisions. By constraining the object's orientation, this term implicitly encourages smoother and more controlled pushing behaviors, reducing impulsive interactions and unnecessary contacts. Furthermore, it improves sim-to-real transfer by mitigating unstable contact patterns, leading to more predictable and robust manipulation in real-world scenarios. $r_{\text{action rate}}$ and $r_{\text{action limits}}$ encourage smooth actions and regular control behavior, reducing abrupt changes in joint commands and helping the robot maintain stable and predictable motions throughout the pushing process.

Terminal reward: When the object's positional error is less than $0.05\,\text{m}$ and its orientation error within $5°$, the task is considered a success. A fixed terminal reward $r_{\text{termination}}$ of $+10.0$ is then applied to provide a stronger learning signal, amplifying the policy gradient for successful steps.

B. Whole-Body Loco-manipulation Controller

We adopt the DreamWaQ [26] architecture for our *Controller*, enabling smooth tracking of the *Planner*'s commands. In particular, we leverage its estimator module to encode a history of proprioceptive observations into a compact latent representation. We detail our RL formulation below:

1) Observations & Action Space: The *Controller* actor network takes as input the observation vector $o_t^c \in \mathbb{R}^{69}$, together with latent variables $v_t \in \mathbb{R}^3$ and $z_t \in \mathbb{R}^{16}$, which are outputs of the estimator given a proprioceptive history $o_{t-10:t}^c$. The estimator is trained jointly with the policy in an end-to-end manner. There, v_t represents the estimated base linear velocity, while z_t represents a latent variable for understanding terrain properties. The actor finally outputs the desired joint positions $a_t^c \in \mathbb{R}^{18}$.

TABLE III

CONTROLLER REWARD TERMS AND WEIGHTS.

Reward	Expression	Weight
$r_{\text{tracking},xy}$	$\exp\left(-\|v_{xy}^{cmd} - v_{xy}\|^2/\sigma_{xy}\right)$	$2dt$
$r_{\text{tracking},z}$	$\exp\left(-\|\omega_z^{cmd} - \omega_z\|^2/\sigma_z\right)$	$1dt$
$r_{\text{tracking},p}$	$\sum_{i\in\{l,r\}} \exp\left(-\|p_i^{cmd} - p_i\|^2/\sigma_p\right)$	$2dt$
$r_{\text{leg lift}}$	$\sum_{i\in\{l,r\}} \mathbb{I}(F_{z,i} < F_{\text{lift}})$	$0.5dt$
$r_{\text{joint deviation}}$	$\|q_t - q_{default}\|^2$	$-0.3dt$
$r_{\text{joint acc}}$	$\|(\dot{q}_t - \dot{q}_{t-1})/dt\|^2$	$-2.5 \times 10^{-7}dt$
r_{torque}	τ_t^2	$-1 \times 10^{-5}dt$
$r_{\text{action rate}}$	$\|a_t^c - a_{t-1}^c\|^2$	$-0.03dt$
$r_{\text{velocity penalty}}$	v_z^2	$-2dt$
$r_{\text{orientation}}$	$\|g_{xy}\|^2$	$-20dt$

2) Reward Functions: To ensure stable execution of our *Controller*, we design the reward structure summarized in Table III. The rewards terms are grouped into two categories:

Tracking: $r_{\text{tracking},xy}$ and $r_{\text{tracking},z}$ encourage the robot to follow the desired velocity, while $r_{\text{tracking},p}$ and $r_{\text{leg lift}}$ guide the two front feet toward their respective target positions.

Regularization: $r_{\text{joint deviation}}$ promotes the rapid development of well-structured motion patterns during early training, which accelerates policy convergence. The remaining reward terms collectively encourage stable body posture, smoother joint motions, and energy-efficient behavior. These regularization terms improve robustness and control stability during both locomotion and manipulation.

C. Training Details

We train our *Controller* and *Planner* using the proximal policy optimization (PPO) [27], a widely used RL algorithm that supports stable and efficient policy optimization.

1) Controller: To avoid unstable behaviors caused by ill-posed target commands during training, we impose explicit constraints on both the velocity commands and the target foot positions. The commanded body velocity is bounded within $[-0.5,\ 0.5]$ m/s along the x axis, $[-0.3,\ 0.3]$ m/s along the y axis, and $[-1,\ 1]$ rad/s in yaw. The target positions of the two front feet are constrained around their default poses. For each foot, the offset from its default pose is limited to $[-0.25, 0.25]$ m, $[-0.2, 0.2]$ m, and $[-0.2, 0.2]$ m along the x-, y-, and z-axes of the body frame, respectively. Additionally, to prevent ill-posed or overlapping references, we enforce the constraint $p_{\text{rf},y} < p_{\text{lf},y} - 0.15$, ensuring sufficient separation between the right and left foreleg targets along the y axis. We apply domain randomization over key physical parameters, as summarized in Table IV. The nominal gains for k_p and k_d are set to 100 and 2.0, respectively.

It is worth noting that our physical robot is equipped with Series Elastic Actuators (SEAs), in which torque is transmitted through an elastic element placed in series with the drivetrain. This introduces compliance and additional actuator dynamics (e.g., spring-mass behavior and force control loops) that are not captured by simple PD-based rigid-actuator approximations, thereby increasing the sim-to-real gap. To improve robustness in the real system and mitigate this gap, we first train the policy using randomized settings in Table IV. The actuator data is then collected from comprehensive locomotion-only roll-outs to avoid ambiguity arising from contact-dependent torque variations. A total of 12 actuator-specific datasets are obtained for the 4 legs undergoing complex locomotion, and used to train 12 independent actuator networks. Consistent with the methodology of Hwangbo et al. [28], each actuator network is implemented as a supervised MLP that maps short histories of joint position error and velocity to the corresponding measured torque. These learned networks then replace the PD-based control for the locomotion legs during policy training, improving the fidelity of actuator dynamics in simulation and enabling more reliable sim-to-real transfer.

TABLE IV

DOMAIN RANDOMIZATION PARAMETERS.

Parameter	Range	Unit
Friction coefficient	$U(0.4,\ 1.2)$	–
COM shift x	$U(-0.2,\ 0.1)$	m
COM shift y	$U(-0.1,\ 0.1)$	m
COM shift z	$U(-0.1,\ 0.1)$	m
Link mass scale	$U(0.9,\ 1.1)$	–
Inertia scale	$U(0.9,\ 1.1)$	–
k_p, k_d scale	$U(0.9,\ 1.1)$	–

TABLE V

OBJECT PARAMETER RANDOMIZATION RANGES.

Parameter	Range	Unit	Parameter	Range	Unit
Length	$U(0.3,\ 0.5)$	m	Mass	$U(5.0,\ 15.0)$	kg
Width	$U(0.25,\ 0.5)$	m	Friction	$U(0.3,\ 0.8)$	–
Height	$U(0.45,\ 0.60)$	m	Object type	[Box, Cylinder]	–

2) Planner: During training of the *Planner*, we freeze the parameters of the *Controller* to ensure reliable execution of motor commands. We randomize the initial poses of both the robot and the object, as well as the target object poses, within a $6\,\text{m} \times 6\,\text{m}$ workspace. Each parallel environment is equipped with a versatile object generator that randomizes physical properties, as summarized in Table V, with boxes and cylinders sampled at a 1:1 ratio. The ground friction coefficient is uniformly sampled from $[0.3,\ 1.0]$.

To reduce the exploration difficulty in early training, we adopt a command-based curriculum that progressively increases task complexity. Directly sampling targets from the full workspace and orientation range often leads to ambiguous learning signals, causing the robot to remain stationary after contact or fail in long-range pushing and large-angle alignment. Specifically, we discretize the target displacement along the y-axis (within a $6\,\text{m}$ range) and the angle (within $\pm 180^\circ$) into 5 curriculum levels. The curriculum levels for translation (y) and orientation (yaw) are adjusted independently based on task performance, measured by an exponential moving average (EMA) of success.

Additionally, to prevent the *Planner* from producing unstable manipulation behaviors, as well as redundant interactions, we introduce a smoothness regularization term inspired by L2C2 [29]. For two consecutive states o_t^c and o_{t+1}^c, a virtual state $\bar{o}_t^c = o_t^c + u(o_{t+1}^c - o_t^c)$ is interpolated with $u \sim \mathcal{U}(0, 1)$.

TABLE VI

SUCCESS RATE AND AVERAGE COMPLETION TIME EVALUATED ACROSS 1000 SIMULATIONS.

Method	Success Rate(%)	Average Time(s)
HeLoM w/o curriculum	49.3	21.4s
HeLoM w/o Key reward	63.7	18.31s
HeLoM w/o λ_{turn} & λ_{push}	76.5	16.9s
HeLoM w/o smoothness loss	89.6	17.2s
HeLoM	**95.6**	**12.5s**

TABLE VII

TRACKING COMPARISON ACROSS COMMAND RANGES.

Metric	Ranges	HeLoM $\mu_{\lvert e\rvert}$ $[\min(e), \max(e)]$	PD only	AcNet only
Linear velocity	[-0.2, 0.2] m/s	**0.035 [-0.084, 0.106]**	0.044 [-0.144, 0.128]	0.040 [-0.171, 0.174]
	[-0.4, 0.4] m/s	**0.044 [-0.138, 0.157]**	0.067 [-0.176, 0.185]	-
Angular velocity	[-0.50, 0.50] rad/s	**0.105 [-0.220, 0.256]**	0.146 [-0.330, 0.311]	0.204 [-0.383, 0.357]
	[-0.75, 0.75] rad/s	**0.128 [-0.282, 0.302]**	0.164 [-0.357, 0.429]	-
Foot tracking	x [0.475, 0.725] m	**0.022 [-0.081, 0.016]**	0.039 [-0.018, 0.103]	0.048 [-0.101, 0.090]
	y [-0.300, 0.300] m	**0.010 [-0.024, 0.025]**	0.020 [-0.021, 0.024]	0.043 [-0.088, 0.190]
	z [-0.075, 0.075] m	**0.008 [-0.011, 0.012]**	0.010 [-0.018, 0.026]	0.070 [-0.122, 0.051]

The smoothness loss is

$$\mathcal{L}_{\text{smooth}} = \lambda_\pi \, D(\pi_\theta(s_t), \pi_\theta(\bar{s}_t)) + \lambda_V \sum_i D\big(V_{\phi_i}(s_t), V_{\phi_i}(\bar{s}_t)\big) \quad (3)$$

where D denotes a distance metric, π_θ the actor, and V_{ϕ_i} the critics. This regularization enforces consistent outputs under minor state perturbations and reduces action jitter.

IV. EXPERIMENTS

We build our training environment in the Isaac Gym simulator and utilize one RTX 4090 GPU to train 4096 robot instances in parallel, which significantly accelerates the convergence of the RL process. We then deploy the trained policy on a **HEBI Daisy** modular series-elastic hexapod robot, to validate performance in the real world.

A. Simulation Results and Success Rate

We randomly sample 1000 simulated interaction environments, under the training parameter range described in Section III-C.2. The physical properties of the objects are summarized in Table V. To quantitatively evaluate the effectiveness of **HeLoM**, we consider two metrics: *success rate* and *average completion time*. Task success is defined as pushing the object to the target within 25 s, with distance $d < 0.05$ m and angular error $\theta_z < 5°$. Any trial exceeding this time limit is regarded as a failure. We compare the performance of our method with the following baselines:

- **HeLoM w/o curriculum**, where we removed the command-based curriculum to evaluate the role of progressive task scaling in enabling learning over a large task space.
- **HeLoM w/o Key reward**, where we removed the rotation guidance term $r_{\text{obj,turn}}$ to evaluate its contribution to large-angle manipulation capability.
- **HeLoM w/o λ_{turn} & λ_{push}**, where we removed the phase-dependent weighting between rotation and translation to evaluate the effect of staged task decomposition on coordinating these objectives.
- **HeLoM w/o smoothness loss**, where we removed the smoothness regularization to evaluate its impact on efficiency and behavioral stability.

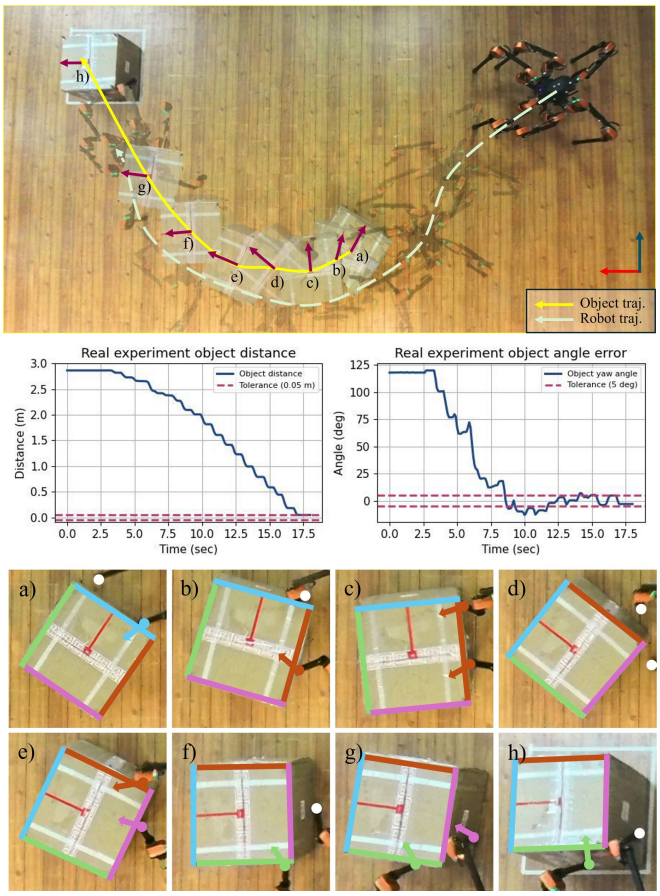
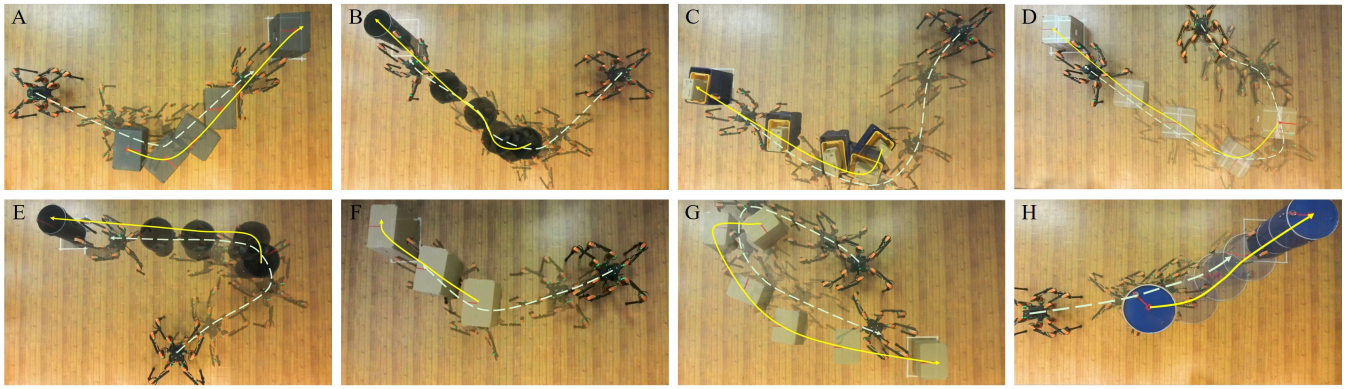


Fig. 3. **Complete robot–object interaction process in a real-world trial**. Top: Robot and object trajectories with intermediate states (a–h).Middle: Object–goal distance and yaw error over time. Bottom: Snapshots of the manipulation process, white dots indicate no contact, while colored arrows denote both the contact surface and the direction of foot motion.

As shown in Table VI, **HeLoM** achieves the highest success rate of 95.6% in these scenarios, while also attaining the shortest average completion time. Failures are mainly observed in lightweight, low-friction cases, where overshooting may occur due to excessive momentum. In contrast, **HeLoM w/o Curriculum** yields the lowest success rate of only 20%. Without progressive task scaling, the policy must directly learn over a large and diverse task space. This makes exploration highly challenging and often leads the policy to local optima. As a result, the robot only acquires simple contact and pushing behaviors, and is limited to straightforward, near-linear tasks. For **HeLoM w/o $r_{\text{obj,turn}}$**, removing the rotation guidance reward significantly degrades performance in tasks requiring large orientation changes. When the orientation error is large, relying solely on the orientation reward provides insufficient guidance for learning effective rotational manipulation behaviors. For **HeLoM w/o λ_{turn} & λ_{push}**, removing the staged weighting between rotation and translation introduces conflicts between the two objectives. For instance, rotating the object to reduce the orientation error may temporarily increase its distance to the target, leading to inconsistent learning signals. This often causes the robot to deviate from the desired trajectory or fail to reach the target pose. Finally, **HeLoM w/o smoothness loss** tends to generate redundant and inefficient manipulation



Case	Mass(kg)	Material	$\Delta\theta_z$(deg)	d(m)	Time (s)	Case	Mass(kg)	Material	$\Delta\theta_z$(deg)	d(m)	Time (s)
A	12.5	Foam	180	4.1	19.2s	E	10.1	Plastic	150	3.60	18.6s
B	10.1	Plastic	110	3.75	17.5s	F	11.4	Cardboard	10	1.8	9.8s
C	13.5	Plastic	165	3.65	20.3s	G	8.65	Cardboard	160	4.34	19.3s
D	7.4	Cardboard	173	4.71	18.3s	H	17.9	Plastic	35	2.35	17.4s

Fig. 4. **Eight representative scenarios from real-world experiments**. The figure illustrates how the hexapod robot uses its legs to push objects with different initial poses, sizes, weights, materials, shapes, and centers of mass to the desired pose. The table below summarizes the object specifications and experimental parameters.

behaviors, significantly increasing the average completion time. The robot often performs unnecessary adjustments or erratic motions, which destabilize object movement and prolong the alignment process.

B. Hardware Validation

1) Loco-manipulation performance: In our real-world experiments, we first evaluated the locomotion performance and manipulation accuracy of our *Controller*. To further validate the effectiveness of the controller design, we compare it with two baseline methods. Table VII illustrates the tracking performance in both commanded velocity and foot position.

The results show that the proposed controller, which combines ACNet for locomotion and a PD controller for manipulation, achieves higher accuracy across all metrics compared to a pure PD controller. In contrast, a pure ACNet-based controller performs significantly poorer in manipulation accuracy and is unusable for high-velocity motions due to aggressive jittering in the front legs. This is consistent with prior work [30], which finds that sim-to-real transfer is highly sensitive to controller structure, and that controllers unsuited to the specific task can produce stiff or unstable behaviour. Therefore, PD control is retained for manipulation, where explicit position regulation is required, while ACNet is applied only to the locomotion legs where accurate actuator modelling is more beneficial.

2) Legs Interaction with Unknown Objects: For our pushing tasks, we employed an external optical motion capture system to acquire the accurate poses of both the robot and the object in the world frame. The current and target pose of the object is transformed into the robot's local body frame and used as input to the policy.

Fig. 3 illustrates the motion states and interaction process of the robot when pushing an unknown object. Throughout the process, the robot relies only on the perceived relative poses of the object and the target (in the robot frame), together with proprioceptive feedback, to guide its leg-driven interactions and accomplish the pushing task. In particular, as shown in Fig. 3(a–h), the robot further switches contact surfaces at key moments and adjust both the contact locations and motion directions, enabling continuous correction of the object's orientation during pushing. This sequence of actions demonstrates the robot's ability to adaptively select interaction strategies when pushing unknown objects, thereby achieving stable and efficient manipulation. Moreover, we observe that, benefiting from coordinated multi-limb interaction, the robot can accomplish efficient object relocation with only minor adjustments to its own posture, which can be clearly observed from the smooth robot trajectory. Meanwhile, both the object–goal distance and yaw error decrease steadily over time and converge within tight tolerances ($d < 0.05\,\text{m}$ and $\theta_z < 5°$).

3) Whole-body Pushing Performance and Robustness: To demonstrate the generalization capability of **HeLoM**, we select eight representative scenarios, as shown in Fig. 4. The robot is initialized at arbitrary positions and orientations to push boxes and cylinders from different directions, including forward, backward, and lateral pushing. Our experimental results show that **HeLoM** can effectively handle objects with varying heights, weights, and materials. Even in challenging cases such as Fig. 4(a), where the center of mass is significantly shifted toward a corner, resulting in uneven mass and friction distribution, the robot can still complete the task with stable performance. Case (d) further involves an uneven surface not encountered during training, which requires precise foot placement to maintain effective contact. If the positioning is not well executed, the foot may become stuck or lose effective contact, which makes the pushing process significantly more challenging. Furthermore, we also evaluate objects beyond the training distribution in terms of weight and size, as shown in Fig. 4(h). In addition, we conduct a load-adaptive whole-body pushing test to highlight the capability of **HeLoM** in coordinating whole-body behaviors. As the load increases, the robot lowers its body and adjusts

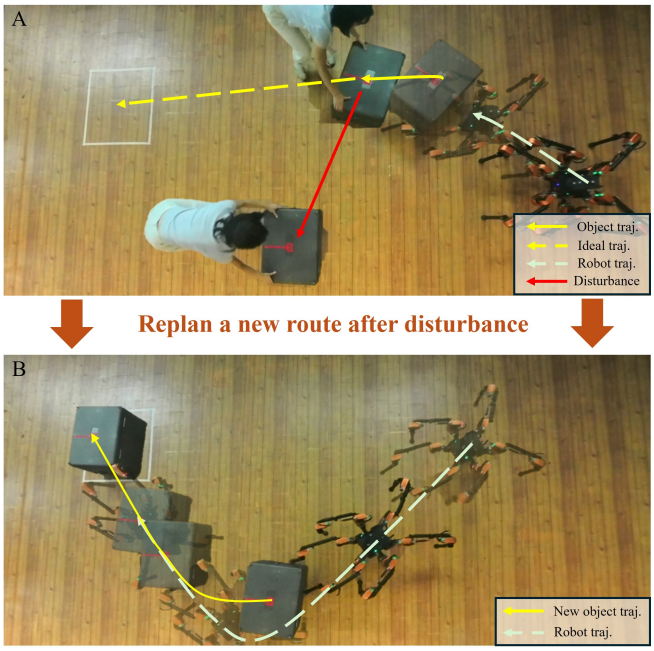


Fig. 5. **Human intervention introduced during the robot pushing task**. (A) A disturbance is introduced by manually shifting the object away from its intended trajectory, forcing the robot to deviate from its current path. (B) The robot adapts by replanning a new route, coordinating its forelegs and body to realign the object and guide it back toward the target position.

its posture to maintain stable and continuous pushing. More details can be found in the accompanying video.

To further evaluate the robustness of the proposed policy in uncertain environments, we introduced external disturbances during the robot's pushing process to reflect real-world variability. As shown in Fig. 5, we manually perturbed the object's position and orientation while the robot was executing the task. The robot adapted its strategy, replanned its actions, and still completed the task successfully. These results show the strong adaptability and resilience of our proposed framework in handling unexpected disturbances.

V. Conclusion

In this paper, we introduce **HeLoM**, a whole-body manipulation framework for hexapod robots. By fully leveraging all legs, our framework enables stable and robust interactions with a wide range of objects. **HeLoM** enables the robot to achieve coordinated whole-body behaviors by adjusting its body posture together with the positions and motion directions of its feet, allowing it to generate effective pushing actions that drive the object toward the desired pose. However, our current implementation relies on an external motion capture system for precise state feedback, which limits its use to controlled environments. Integrating onboard sensing modules such as cameras and LiDAR may allow us to overcome these constraints and enable deployment to real-world scenarios. That being said, we believe that the results of this study not only validate the effectiveness and potential of multi-legged robots as stable and versatile manipulation platforms but also demonstrate that decomposing whole-body manipulation tasks into planning and control provides a clear task allocation and an efficient execution mechanism for solving complex manipulation problems.

References

[1] Y. Gong, G. Sun, A. Nair, A. Bidwai, R. CS, J. Grezmak, G. Sartoretti, and K. A. Daltorio, "Legged robots for object manipulation: A review," *Frontiers in Mechanical Engineering*, vol. 9, p. 1142421, 2023.

[2] N. Dengler, D. Großklaus, and M. Bennewitz, "Learning goal-oriented non-prehensile pushing in cluttered scenes," in *Proc. of IROS 2022*.

[3] S. Jeon, M. Jung, S. Choi, B. Kim, and J. Hwangbo, "Learning whole-body manipulation for quadrupedal robot," *IEEE Robotics and Automation Letters (RA-L)*, vol. 9, no. 1, pp. 699–706, 2023.

[4] L. Peizhuo, H. Li, G. Sun, C. Jin, X. Yang, G. Bellegarda, M. Shafiee, Y. Cao, A. Ijspeert, and G. Sartoretti, "SATA: Safe and Adaptive Torque-based locomotion policies inspired by Animal learning," *Robotics: Science and Systems XXI*, 2025.

[5] T. Portela, A. Cramariuc, M. Mittal, and M. Hutter, "Whole-body end-effector pose tracking," in *Proc. of ICRA 2025*, pp. 11 205–11 211.

[6] I. Dadiotis, M. Mittal, N. Tsagarakis, and M. Hutter, "Dynamic object goal pushing with mobile manipulators through model-free constrained reinforcement learning," in *Proc. of ICRA 2025*, pp. 13 363–13 369.

[7] J. Cheng, D. Kang, G. Fadini, G. Shi, and S. Coros, "Rambo: Rl-augmented model-based whole-body control for loco-manipulation," *IEEE Robotics and Automation Letters*, 2025.

[8] B. Leung, S. Gorb, and P. Manoonpong, "Nature's all-in-one: Multitasking robots inspired by dung beetles," *Advanced Science*, vol. 11, no. 47, p. 2408080, 2024.

[9] M. Sombolestan and Q. Nguyen, "Hierarchical adaptive loco-manipulation control for quadruped robots," *arXiv preprint arXiv:2209.13145*, 2022.

[10] G. Xin, F. Zeng, and K. Qin, "Loco-manipulation control for arm-mounted quadruped robots: Dynamic and kinematic strategies," *Machines*, vol. 10, no. 8, p. 719, 2022.

[11] H. Ferrolho, V. Ivan, W. Merkt, I. Havoutis, and S. Vijayakumar, "Roloma: Robust loco-manipulation for quadruped robots with arms," *Autonomous Robots*, vol. 47, no. 8, pp. 1463–1481, 2023.

[12] M. P. Polverini *et al.*, "Multi-contact heavy object pushing with a centaur-type humanoid robot: Planning and control for a real demonstrator," *RA-L*, vol. 5(2), no. 2, pp. 859–866.

[13] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the mit cheetah 3 through convex model-predictive control," in *Proc. of IROS 2018*, 2018, pp. 1–9.

[14] Y. Ji, G. B. Margolis, and P. Agrawal, "Dribblebot: Dynamic legged manipulation in the wild," *arXiv preprint arXiv:2304.01159*, 2023.

[15] Z. Fu, X. Cheng, and D. Pathak, "Deep whole-body control: learning a unified policy for manipulation and locomotion," in *Conference on Robot Learning*. PMLR, 2023, pp. 138–149.

[16] T. Portela, G. B. Margolis, Y. Ji, and P. Agrawal, "Learning force control for legged manipulation," in *Proc. of ICRA 2024*.

[17] X. Zhu, Y. Chen, L. Sun, F. Niroui, S. Le Cleac'h, J. Wang, and K. Fang, "Versatile loco-manipulation through flexible interlimb coordination," in *Proc. of CoRL 2025*, pp. 610–632.

[18] H. Ha, Y. Gao, Z. Fu, J. Tan, and S. Song, "Umi on legs: Making manipulation policies mobile with manipulation-centric whole-body controllers," *arXiv preprint arXiv:2407.10353*, 2024.

[19] G. Sun and G. Sartoretti, "Joint-space cpg for safe foothold planning and body pose control during locomotion and climbing," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 9889–9896, 2022.

[20] B. Lu, B. Tam, and N. Kottege, "Autonomous obstacle legipulation with a hexapod robot," *arXiv preprint arXiv:2011.06227*, 2020.

[21] K. Xu, R. Qin, C. Chen, G. Dong, J. Chen, and X. Ding, "Design and multimodal locomotion plan of a hexapod robot with improved knee joints," *Journal of Field Robotics*, vol. 41(5), pp. 1279–1296, 2024.

[22] Z. Zhang, L. Ding, P. Xu, T. Liu, A. Zhang, H. Yang, H. Gao, and Z. Deng, "Legs-as-arms: A multitask reinforcement learning framework for simultaneous loco-manipulation in hexapod robots," *IEEE Transactions on Industrial Electronics*, 2024.

[23] A. Rigo, Y. Chen, S. K. Gupta, and Q. Nguyen, "Contact optimization for non-prehensile loco-manipulation via hierarchical model predictive control," *arXiv preprint arXiv:2210.03442*, 2022.

[24] P. Arm, M. Mittal, H. Kolvenbach, and M. Hutter, "Pedipulate: Enabling manipulation skills using a quadruped robot's leg," in *Proc. of ICRA 2024*, pp. 5717–5723.

[25] Y. Feng, C. Hong, Y. Niu, S. Liu, Y. Yang, and D. Zhao, "Learning multi-agent loco-manipulation for long-horizon quadrupedal pushing," in *Proc. of ICRA 2025*, pp. 14 441–14 448.

[26] I. M. A. Nahrendra, B. Yu, and H. Myung, "Dreamwaq: Learning robust quadrupedal locomotion with implicit terrain imagination via deep reinforcement learning," in *Proc. of ICRA 2023*, pp. 5078–5084.
[27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
[28] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.
[29] T. Kobayashi, "L2c2: Locally lipschitz continuous constraint towards stable and smooth reinforcement learning," in *Proc. of IROS 2022*, pp. 4032–4039.
[30] Z. Xie, X. Da, M. van de Panne, B. Babich, and A. Garg, "Dynamics randomization revisited: A case study for quadrupedal locomotion," in *Proc. of ICRA 2021*, pp. 4955–4961.